

DST Airlines

Membre de l'équipe

Nicolas CALO
Johan CLOOS
Younes Es-soualhi
Rathana LAT
Youssef ZNATI

Étape 2 - Architecture des données

Description

Objectif

- Il y a plusieurs options qui s'offrent à nous. Lors de la précédente étape, nous avons observé qu'il y a plusieurs "types" de données. Nous allons qualifier des données de fixe comme les informations sur les aéroports et de variables, les informations sur les vols.
- Cette diversification de données va conduire à une utilisation de différentes bases de données.

Modules / Masterclass / Templates

- SQL
- SnowFlake
- DBT
- Elasticsearch
- MongoDB
- Neo4j

Conditions de validation du projet

- Une base de données relationnelle
- Diagramme UML
- Un fichier de requête SQL pour montrer que c'est bien fonctionnel
- Même rendu mais exemples de requêtes Elastic/Mongo

Base de données relationnelle

Sous Postgresql, les tables des données de référence statiques ont été créées.
L'alimentation de ces tables proviennent d'extractions de données par webscraping.

Nom des tables :

- Continent
- Subcontinent
- Country
- Location
- Airport
- Iata
- Icao

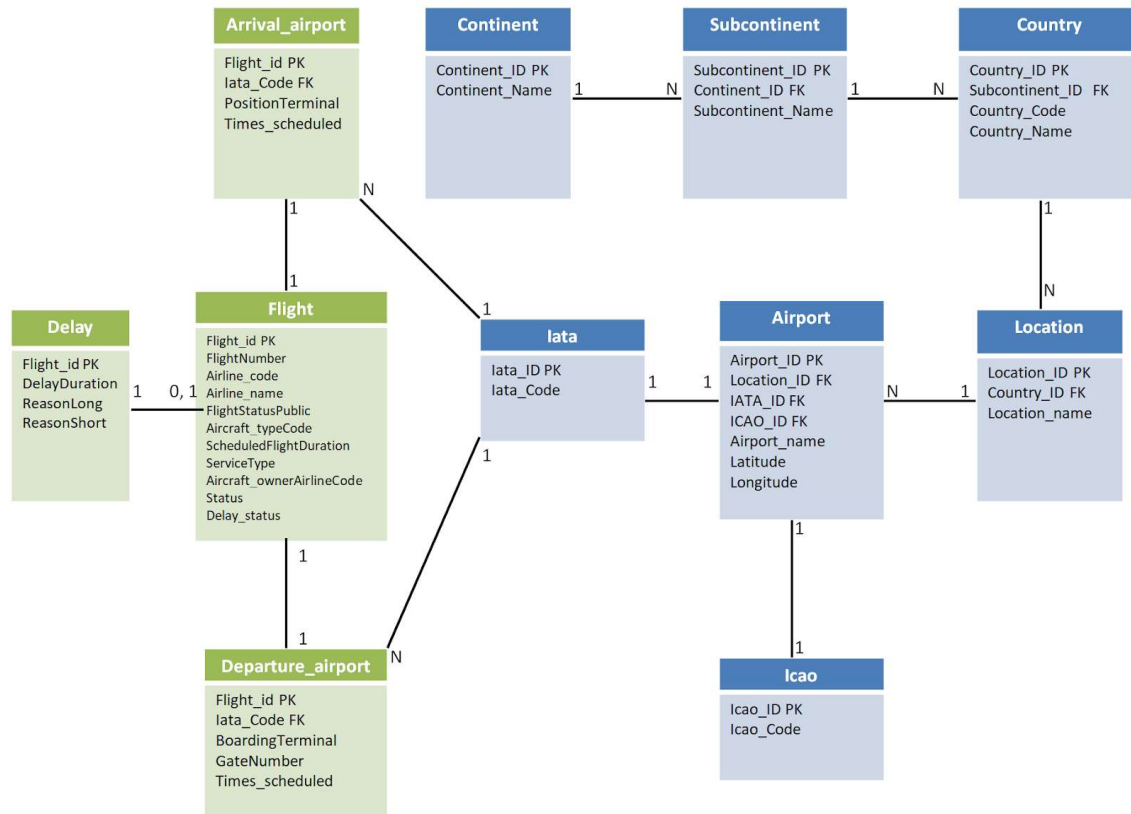
Les données stockées dans MongoDB sont des fichiers au format json provenant de l'API d'Air France KLM contenant les informations concernant les vols. Les tables des données exportées de MongoDB ont été créées.

Nom des tables :

- Flight
- Arrival_airport
- Departure_airport
- Delay

Diagramme UML

Les tables en bleu concernant les données de référence statique. Les tables en vert concernent les données extraites de MongoDB.



Un fichier de requête SQL pour montrer que c'est bien fonctionnel

Script de création de table de référence statique

```
DROP TABLE IF EXISTS Continent;
CREATE TABLE Continent (
  Continent_ID serial PRIMARY KEY,
  Continent_Name varchar(50) NOT NULL
);

DROP TABLE IF EXISTS Subcontinent;
CREATE TABLE Subcontinent (
  Subcontinent_ID serial PRIMARY KEY,
  Subcontinent_Name varchar(100) NOT NULL,
  Continent_ID int DEFAULT NULL
);
```

```

DROP TABLE IF EXISTS Country;
CREATE TABLE Country (
Country_ID serial PRIMARY KEY,
Country_Name varchar(100) DEFAULT NULL,
Country_Code varchar(100) NOT NULL,
Subcontinent_ID int DEFAULT NULL
);

```

```

DROP TABLE IF EXISTS Location;
CREATE TABLE Location (
Location_ID serial PRIMARY KEY,
Location_name varchar(100) NOT NULL,
Country_ID int DEFAULT NULL
);

```

```

DROP TABLE IF EXISTS Airport;
CREATE TABLE Airport (
Airport_ID serial PRIMARY KEY,
Airport_name varchar(100) NOT NULL,
Latitude float NOT NULL,
Longitude float NOT NULL,
Location_ID int DEFAULT NULL,
Iata_ID int DEFAULT NULL,
Icao_ID int DEFAULT NULL
);

```

```

DROP TABLE IF EXISTS Iata;
CREATE TABLE Iata (
Iata_ID serial PRIMARY KEY,
Iata_Code varchar(10) NOT NULL
);

```

```

DROP TABLE IF EXISTS Icao;
CREATE TABLE Icao (
Icao_ID serial PRIMARY KEY,
Icao_Code varchar(10) NOT NULL
);

```

Script de création de table des données de vol extraite de MongoDB

```

DROP TABLE IF EXISTS flight;
CREATE TABLE flight (
flight_id varchar(50) NOT NULL,
flightNumber int DEFAULT NULL,
airline_code varchar(50) NOT NULL,
airline_name varchar(100) NOT NULL,
flightStatusPublic varchar(50) NULL,
flightLegs_aircraft_typeCode varchar(50) NULL,
flightLegs_scheduledFlightDuration varchar(50) NULL,
flightLegs_serviceType varchar(50) NULL,
flightLegs_aircraft_ownerAirlineCode varchar(50) NULL,
flightLegs_status varchar(50) NULL,
delay_status boolean NULL
);

```

```
);
```

```
DROP TABLE IF EXISTS arrival_airport;  
CREATE TABLE arrival_airport (  
    flight_id varchar(50) NOT NULL,  
    flightLegs_arrInfo_airport_code varchar(50) NOT NULL,  
    flightLegs_arrInfo_airport_places_arrivalPositionTerminal varchar(50) NULL,  
    flightLegs_arrInfo_times_scheduled varchar(50) NULL  
);
```

```
DROP TABLE IF EXISTS departure_airport;  
CREATE TABLE departure_airport (  
    flight_id varchar(50) NOT NULL,  
    flightLegs_depInfo_airport_code varchar(50) NOT NULL,  
    flightLegs_depInfo_airport_places_depPosTerm_boardingTerminal varchar(50)  
    NULL,  
    flightLegs_depInfo_airport_places_depPosTerm_gateNumber varchar(50) NULL,  
    flightLegs_depInfo_times_scheduled varchar(50) NULL  
);
```

```
DROP TABLE IF EXISTS delay;  
CREATE TABLE delay (  
    flight_id varchar(50) NOT NULL,  
    flightLegs_irregularity_delayDuration varchar(250) NULL,  
    flightLegs_irregularity_delayInfo_delayReasonPublicLong varchar(250) NULL,  
    flightLegs_irregularity_delayInfo_delayReasonPublicShort varchar(250) NULL  
);
```

Requête fonctionnelle

```
select distinct(flight.flight_id), Location.Location_name,  
Airport.Airport_name, Iata.Iata_Code, Icao.Icao_Code,  
flightLegs_depInfo_times_scheduled,  
departure_airport.flightlegs_depinfo_airport_places_depposterm_boardingterm  
inal,  
departure_airport.flightLegs_depInfo_airport_places_depPosTerm_gateNumber  
from Location Location  
INNER JOIN Airport Airport ON Airport.Location_ID = Location.Location_ID  
INNER JOIN Iata Iata ON Iata.Iata_ID = Airport.Iata_ID  
INNER JOIN Icao Icao ON Icao.Icao_ID = Airport.Icao_ID  
INNER JOIN departure_airport departure_airport ON  
departure_airport.flightLegs_depInfo_airport_code = Iata.Iata_code  
INNER JOIN flight flight ON flight.flight_id = departure_airport.flight_id  
INNER JOIN delay delay ON delay.flight_id = flight.flight_id  
where Location.Location_name = 'Ile-de-France';
```

12

select distinct (flight.flight_id), Location.Location_name, Airport.Airport_name, Iata.Iata_Code, Icao.Icao_Code,

13

flightlegs_depInfo_times_scheduled, departure_airport.flightlegs_depinfo_airport_places_depposTerm_boardingterminal,

14

departure_airport.flightlegs_depInfo_airport_places_depPosTerm_gateNumber

15

from Location Location

16

INNER JOIN Airport Airport ON Airport.Location_ID = Location.Location_ID

17

INNER JOIN Iata Iata ON Iata.Iata_ID = Airport.Iata_ID

18

INNER JOIN Icao Icao ON Icao.Icao_ID = Airport.Icao_ID

19

INNER JOIN departure_airport departure_airport ON departure_airport.flightlegs_depInfo_airport_code = Iata.Iata_code

20

INNER JOIN flight flight ON flight.flight_id = departure_airport.flight_id

21

INNER JOIN delay delay ON delay.flight_id = flight.flight_id

22

where Location.Location_name = 'Ile-de-France';

23

Données

Messages

Notifications

SQL

Affichage des lignes : 1 à 1000

Page No: 1

de 17

flight_id	location_name	airport_name	iata_code	icao_code	flightlegs_depinfo_times_scheduled	flightlegs_depinfo_places	flightlegs_depinfo_posTerm	flightlegs_depinfo_boardingterminal	flightlegs_depinfo_gateNumber
character varying (50)	character varying (50)	character varying (100)	character varying (3)	character varying (3)	character varying (50)	character varying (1)	character varying (1)	character varying (10)	character varying (5)
439	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-27T10:35:00.000+02:00	3		[12]	
440	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-27T16:10:00.000+02:00	3		[12]	
441	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-27T13:05:00.000+02:00	3		[10]	
442	Ile-de-France	Aéroport de Paris-Orly	ORY	LFPO	2025-05-27T09:55:00.000+02:00	1		[A20]	
443	Ile-de-France	Aéroport de Paris-Orly	ORY	LFPO	2025-05-27T09:05:00.000+02:00	1		[B14]	
444	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-28T22:15:00.000+02:00	2D		[D54]	
445	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-28T21:10:00.000+02:00	2F		[F50]	
446	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-28T21:55:00.000+02:00	2E		[K32]	
447	Ile-de-France	Paris Charles de Gaulle Airport	CDG	LFPG	2025-05-28T20:45:00.000+02:00	2G		[L30]	
Nombre total de lignes : 16107									
Requête terminée 00:00:00.752									

CRLF

Lgn 27, Col 1

Conclusion

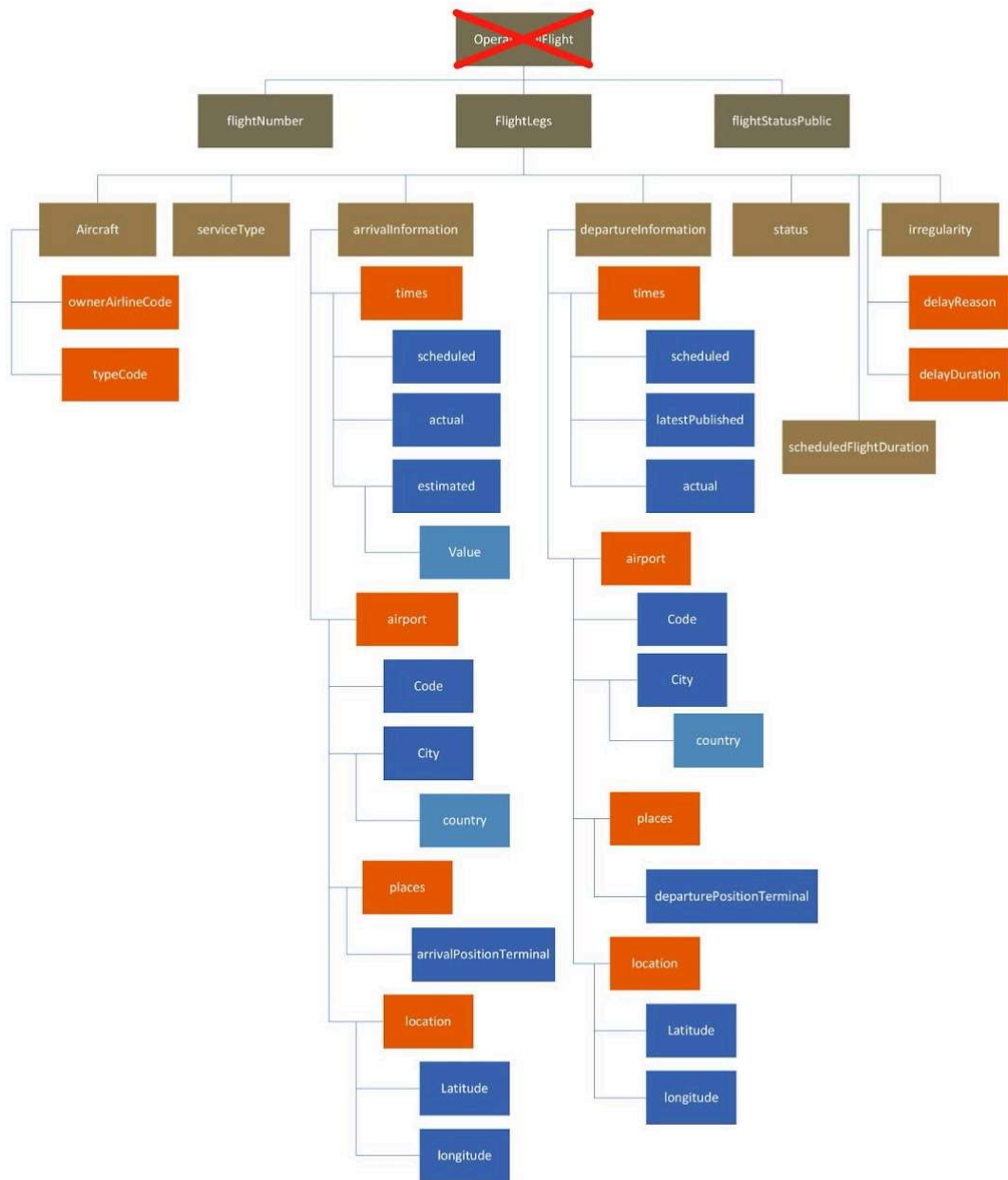
La base de données relationnelle mise en place permettra de faire des requêtes pour récupérer les informations à partir du continent jusqu'à la porte d'embarquement du vol (si elle est disponible).



**La Bdd stockant pour l'heure les objets
"flights" récupérés sur l'API AFKLM
d'Air France**



Schema de l'objet stocké



Note: Les objets flights étaient dans des opérations flights. Cependant ce sont bien des vols que l'on souhaite récupérer, nous avons donc extrait les vols de opération flight pour les stocker dans une collection flight.

Voici un flight telle qu'il est stocker en BDD:

```
{
  "_id": "6900d152be9ad5e05baf37d0",
  "flightNumber": 1219,
  "flightScheduleDate": "2025-06-04",
  "id": "20250604+SK+1219",
  "route": [
    "CPH",
    "AAL"
  ],
  "airline": {
    "code": "SK",
    "name": "SCANDINAVIAN AIRLINES SYSTEM (SAS)"
  },
  "codeShareRelations": [
    {
      "marketingFlightNumber": 4934,
      "code": "F",
      "type": "PASSENGER",
      "airline": {
        "code": "AF",
        "name": "AIR FRANCE"
      }
    },
    {
      "marketingFlightNumber": 8412,
      "airline": {
        "code": "KL",
        "name": "KLM ROYAL DUTCH AIRLINES"
      }
    }
  ],
}
```

```
"flightStatusPublic": "ARRIVED",
"flightStatusPublicLangTransl": "Arrived",
"flightLegs": [
  {
    "publishedStatus": "ARRIVED",
    "departureInformation": {
      "airport": {
        "code": "CPH",
        "name": "KASTRUP AIRPORT",
        "nameLangTranl": "Copenhagen Airport",
        "city": {
          "code": "CPH",
          "name": "COPENHAGEN",
          "nameLangTranl": "Copenhagen",
          "country": {
            "areaCode": "E",
            "code": "DK",
            "name": "DENMARK",
            "nameLangTranl": "Denmark",
            "euroCountry": "N",
            "euCountry": "Y"
          }
        }
      },
      "location": {
        "latitude": 55.6081,
        "longitude": 12.6333
      },
      "places": {
        "gateNumber": [
          "B8"
        ],
        "terminalCode": "3",
        "boardingTerminal": "3",
        "checkInZone": [],
        "paxDepartureGate": []
      }
    }
  }
]
```

```
    }  
  },  
  "times": {  
    "actual": "2025-06-04T19:49:00.000+02:00",  
    "scheduled": "2025-06-04T19:55:00.000+02:00",  
    "latestPublished": "2025-06-04T19:49:00.000+02:00",  
    "actualTakeOffTime": "2025-06-04T19:58:00.000+02:00"  
  },  
},  
"arrivalInformation": {  
  "airport": {  
    "code": "AAL",  
    "name": "AALBORG AIRPORT",  
    "nameLangTranl": "Aalborg",  
    "city": {  
      "code": "AAL",  
      "name": "AALBORG",  
      "nameLangTranl": "Aalborg",  
      "country": {  
        "areaCode": "E",  
        "code": "DK",  
        "name": "DENMARK",  
        "nameLangTranl": "Denmark",  
        "euroCountry": "N",  
        "euCountry": "Y"  
      }  
    }  
  },  
},  
"location": {  
  "latitude": 57.0931,  
  "longitude": 9.85  
},  
"places": {  
  "gateNumber": [],  
  "baggageBelt": [  
    "2"
```

```
    ],
    "arrivalHall": []
  }
},
"times": {
  "actual": "2025-06-04T20:32:00.000+02:00",
  "scheduled": "2025-06-04T20:40:00.000+02:00",
  "latestPublished": "2025-06-04T20:32:00.000+02:00",
  "estimated": {
    "value": "2025-06-04T20:32:00.000+02:00"
  },
  "actualTouchDownTime": "2025-06-04T20:26:00.000+02:00"
}
},
"legStatusPublic": "ARRIVED",
"legStatusPublicLangTransl": "Arrived",
"passengerCustomsStatus": "SCHENGEN",
"serviceType": "J",
"serviceName": "Normal Service",
"restricted": false,
"scheduledFlightDuration": "PT45M",
"departureDateTimeDifference": "PT-6M",
"arrivalDateTimeDifference": "PT-8M",
"timeToArrival": "PT0S",
"completionPercentage": "100",
"timeZoneDifference": "+0000",
"aircraft": {
  "registration": "SEROC",
  "typeCode": "32N",
  "typeName": "AIRBUS A320NEO",
  "ownerAirlineCode": "SK",
  "ownerAirlineName": "SCANDINAVIAN AIRLINES SYSTEM
(SAS)",
  "physicalPaxConfiguration": "C024M156",
  "physicalFreightConfiguration": "PP000LL000",
```

```
    "operationalConfiguration":  
    "CDZJYSBPAEMHQVWURKNLTOG",  
    "cockpitCrewEmployer": "SK",  
    "cabinCrewEmployer": "SK"  
  },  
  "irregularity": {  
    "cancelled": "N",  
    "delayCode": [],  
    "delayReason": [],  
    "delayDuration": [],  
    "delayReasonCodePublic": [],  
    "delayReasonPublicLangTransl": [],  
    "delayReasonPublic": [],  
    "delayInformation": []  
  }  
}  
]  
}
```

Les Outils d'interactions avec la BDD

1 readme

1 .sh qui permet : Démarrage container docker + BDD + Import dump.archive dans la BDD

2 .sh permettant d'importer les json de l'api Airfrance (Simulation de situation d'entreprise:

Un outil pour import pour les personnes internes, et un outil simulant un client externe

qui doit passer à terme par des appels api pour insérer de la data dans la base. Pour le moment, cet outil appelle les use cases, mais à terme il pourrait appeler les routes api qui ne sont pas dev à ce stade du projet).

1 Route api pour sortir un dump en .archive

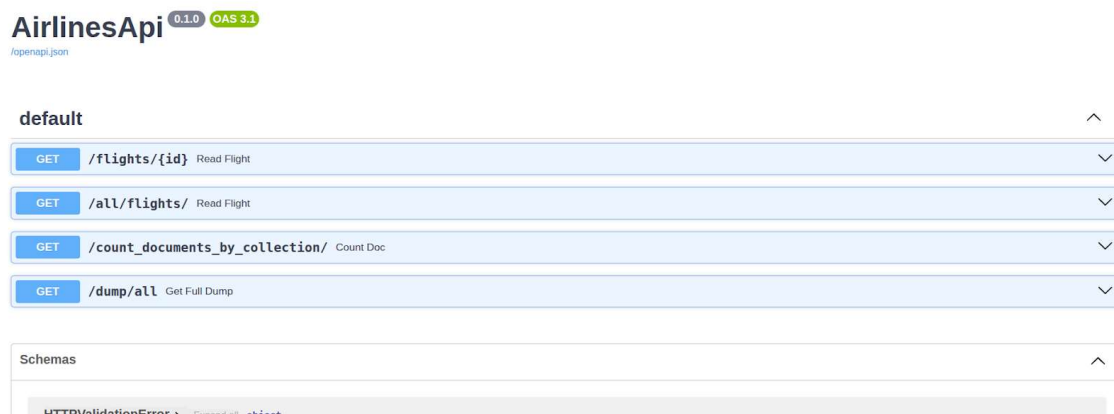
1 Ligne de commande fourni dans le readme pour exporter un dump .archive

1 Ligne de commande fourni dans le readme pour importer un dump .archive

1 Route api pour sortir une version tabulaire .csv de la data.

1 .sh qui fait la même chose

1 Api (routes visibles sur l'image ci dessous, cependant, c'est une première ébauche, il n'a pas encore été décidé si on fait du rest, si on va au bout de cet api, si elle reste seulement un échantillon pour illustrer ce qui pourrait être une bonne pratique avec plus de temps.)



L'intérêt de l'Api: Permettre l'interaction du client avec la BDD dans le respect d'un contrat garantissant la sécurité de la BDD et facilitant l'interaction du client avec la BDD. Il ne peut y avoir d'interaction entre la BDD et le Client qu'en passant par l'API, et l'API garantit que le contrat est respecté.

ARCHITECTURE DU CODE

```
bdd/MongoDb/
├── mongo_db_interaction/
│   ├── main.py
│   ├── SERIALIZER/
│   ├── USE_CASES/
│   ├── SERVICES/
│   ├── REPOSITOIRES/
│   └── DB_CONTEXT/
├── data_dump/
├── docker-compose.yml
├── .env
└── envstruct
```

Il s'agit d'un échantillon de ce que pourrait être une architecture inspirée de l'inversion de dépendance et basée sur la single responsibility.

Il en reste que les use cases sont agnostiques de la BDD. Cependant, prenons l'exemple du use_case insert_flight_uc:

Comme on peut le voir, il est agnostique de la BDD... Qu'est ce qu'il fait?

Il insert un flight, et vérifie si à l'endroit où il insert le flight, (à cette échelle on ignore si c'est dans une Bdd et quelle BDD), si l'id de ce flight est déjà présent.

On voit dans le code joint ci-après, qu'il appelle directement le get_by_id du repository, plutôt qu'utiliser le use case get_by_id développé pour l'api. En fait, à terme, il devrait faire appel au use case get_by_id_flight.

Pourquoi? Parce que le use case get_by_id_flight est aussi censé faire appel à un service qui lui va vérifier si l'id est conforme au format d'id attendu. Ce sont des choses non faites pour le moment, mais des améliorations possibles.

De plus, avec ce type de développement, on peut lire le use case, comprendre ce qu'il fait, sans aller sur la couche du dessous.

Donc si on venait à changer de BDD, sans changer de logique métier, il n'y aurait que les Repositories à modifier. Car on insérerait toujours un flight en appelant le get_by_id, que ce soit pour le rentrer dans une bdd SQL, ou noSQL... Par exemple.

```
from ..REPOSITORIES.flights import insert_one, get_by_id

def insert_flight(flight):
    id = flight['id']

    if is_flight_exit(id) == False:
        insert_one(flight)

def is_flight_exit(id):
    flight_id = get_by_id(id)
    if flight_id == None:
        print(id)
        return False

    else:
        print(id + " Already exist")
        return True
```

I

CONCLUSION:

Les fondations de la BDD MongoDB sont posées, et des outils d'interaction pour les clients de la BDD sont disponibles et utilisables.